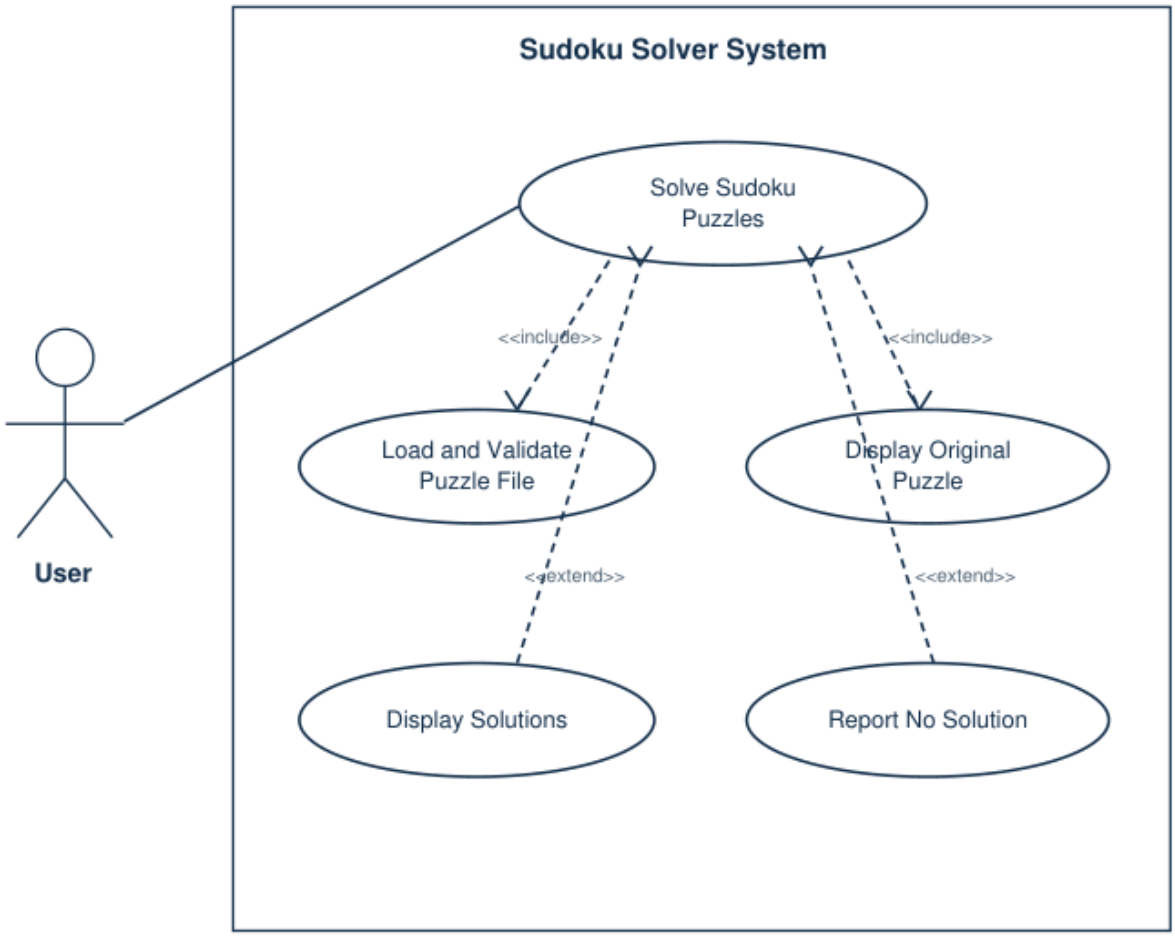
UML Use Case Diagram – Sudoku Solver



The User starts the Sudoku Solver. The system loads and checks each puzzle, displays the original grid, and reports all valid solutions or that no solution exists. The five files are processed automatically.

Use Case Specification

a. Identify / Discover a Use Case

Use case: Solve Sudoku Puzzles
Primary actor: User
System: Sudoku Solver
Goal: Process five Sudoku puzzle files and display every valid solution, or report when no solution exists.

b. Brief Description

The User runs the Sudoku Solver program. The system reads five puzzle files, checks their contents, displays each original puzzle, and searches for all valid solutions. It prints each solution or reports that no solution exists.

c. Step-by-Step Flow (Outline)

1. User starts the Sudoku Solver program.
2. System opens the next puzzle file, from puzzle1.txt through puzzle5.txt.
3. System reads the 9×9 puzzle and checks the file contents.
4. System displays the original puzzle.
5. System checks the starting numbers for conflicts.
6. System fills empty cells using recursion and backtracking.
7. System displays every solution or reports that no solution exists.
8. System repeats for the remaining files and then finishes.

d. Detailed Description

Use case name	Solve Sudoku Puzzles
Primary actor	User
Trigger	User starts the program
Preconditions	Program can run; puzzle files are expected in the working folder
Success result	Original puzzle and all valid solutions are displayed
Other results	No solution found, or an error message for an unavailable or invalid file

Detailed Use Case: Events and Alternatives

Basic Flow of Events

9. **Start program:** The User runs the Sudoku Solver. The system begins with puzzle1.txt.
10. **Open and read file:** The system opens the current file and reads exactly 81 cells, each containing a digit from 1 to 9 or an underscore.
11. **Display puzzle:** The system displays the file name and original 9×9 grid.
12. **Check starting puzzle:** The system checks that the given digits do not repeat in any row, column, or 3×3 block.
13. **Search for solutions:** The system tries valid digits in empty cells using recursion.
14. **Backtrack when needed:** If a choice cannot complete the puzzle, the system removes it and tries another digit.
15. **Show results:** The system prints every complete solution and the total number found, or reports no solution.
16. **Continue and finish:** The system repeats the process for the remaining files and ends after the fifth file.

Alternative Flows

A1. File cannot open: Show an error for that file and continue with the next puzzle.

A2. Invalid file contents: Show an error for missing, extra, or invalid characters and continue.

A3. Conflicting starting digits: Report the invalid puzzle without attempting to solve it.

A4. No valid solution: Display “No solution found” and continue to the next puzzle.

A5. Multiple solutions: Print every valid solution and the total number found.